

ONIT-PNG

Observatoire National Intelligent
des Telecommunications

Guide des Tests End-to-End

Playwright - Couverture complete des API et de l'interface utilisateur

Version 1.0 - 17/05/2026

ARPT - Autorite de Regulation des Postes et Telecommunications de Guinee

Table des Matieres

1. Architecture des Tests E2E
2. Installation et Configuration
3. Donnees de Test (Fixtures)
4. Tests d'Authentification
5. Tests API - Alertes
6. Tests API - Mesures QoS
7. Tests API - Campagnes
8. Tests API - Rapports
9. Tests API - Utilisateurs, Roles et Scores
10. Tests API - Dashboard, Map, QoS et Audit
11. Tests UI - Navigation et Dashboards
12. Tests UI - Carte Interactive SIG
13. Tests RBAC - Controle d'Acces
14. Fichiers de Donnees d'Import
15. Commandes d'Execution
16. Matrice RBAC Complete

1. Architecture des Tests E2E

Les tests end-to-end (E2E) de la plateforme ONIT-PNG sont construits avec **Playwright** et couvrent l'ensemble des fonctionnalités de l'application : API REST, interface utilisateur, carte interactive Leaflet, et contrôle d'accès RBAC. L'architecture suit un modèle en couches qui sépare clairement les fixtures de données, les helpers d'authentification, et les scénarios de test.

1.1 Structure des Fichiers

Fichier	Description
playwright.config.ts	Configuration Playwright (baseUrl, timeouts, reporter)
e2e/fixtures/test-data.ts	Données de test : identifiants, payloads API, codes opérateurs/regions
e2e/fixtures/sample-mesures-import.csv	Fichier CSV exemple pour l'import en masse de mesures
e2e/fixtures/sample-mesures-import.json	Fichier JSON exemple pour l'import en masse de mesures
e2e/helpers/auth.ts	Helpers d'authentification (API + UI)
e2e/helpers/api-helpers.ts	Helpers API (récupérer IDs, créer entités de test, nettoyage)
e2e/auth.spec.ts	Tests d'authentification NextAuth
e2e/api-alerts.spec.ts	Tests CRUD sur /api/alerts
e2e/api-mesures.spec.ts	Tests CRUD sur /api/mesures (unique + bulk JSON/CSV)
e2e/api-campaigns.spec.ts	Tests CRUD sur /api/campaigns
e2e/api-reports.spec.ts	Tests CRUD sur /api/reports
e2e/api-users-roles-scores.spec.ts	Tests sur /api/users, /api/roles, /api/scores
e2e/api-dashboard-map.spec.ts	Tests sur /api/dashboard, /api/map, /api/qos, /api/scoring, /api/audit-logs
e2e/ui-navigation.spec.ts	Tests de navigation UI, login modal, responsive
e2e/ui-map.spec.ts	Tests de la carte Leaflet (découpage CNT, popups, zoom)
e2e/rbac.spec.ts	Tests exhaustifs du contrôle d'accès RBAC par rôle

1.2 Principes de Conception

Les tests E2E suivent plusieurs principes fondamentaux qui garantissent leur fiabilité et leur maintenabilité. Premièrement, chaque suite de test crée son propre contexte d'authentification via l'API NextAuth, ce qui permet de tester les permissions de chaque rôle de manière isolée. Deuxièmement, les données de test sont centralisées dans le fichier test-data.ts, facilitant la maintenance quand les schémas ou les valeurs de référence changent. Troisièmement, chaque test qui crée des entités les nettoie après exécution (résolution d'alertes, désactivation d'utilisateurs, etc.) pour éviter la pollution de la base de données.

- Execution sequentielle (workers=1) pour eviter les conflits SQLite
- Contextes d'authentification independants par role (admin, operateur, etc.)
- Donnees de test reproductibles et centralisees dans les fixtures
- Nettoyage automatique des entites creees pendant les tests
- Screenshots et traces automatiques en cas d'echec

2. Installation et Configuration

L'installation des tests E2E nécessite Playwright et le navigateur Chromium. La configuration est prête à l'emploi avec le fichier `playwright.config.ts` qui définit le serveur de développement, les timeouts, et les options de reporting.

2.1 Installation des Dependances

```
npm install
npx playwright install chromium
```

2.2 Configuration Playwright

Le fichier `playwright.config.ts` est configuré avec les paramètres suivants : base URL sur `localhost:3000`, exécution séquentielle (`workers=1`) pour la cohérence de la base SQLite, timeout de 30 secondes par test, et génération de rapports HTML. Le serveur de développement est lancé automatiquement si nécessaire.

Parametre	Valeur	Justification
baseURL	<code>http://localhost:3000</code>	Serveur de dev Next.js
workers	<code>1</code>	Exécution séquentielle pour SQLite
timeout	<code>30000ms</code>	Délai suffisant pour les appels API
retries	<code>0 (local) / 2 (CI)</code>	Pas de retry en dev, retry en CI
reporter	<code>HTML + list</code>	Rapport détaillé + console
webServer	<code>npm run dev</code>	Démarrage automatique du serveur

3. Donnees de Test (Fixtures)

Le fichier `e2e/fixtures/test-data.ts` contient toutes les donnees de test reutilisables. Il centralise les identifiants de connexion pour les 10 utilisateurs seedes, les codes operateurs et regions, les payloads API complets pour chaque endpoint, et les coordonnees GPS de reference. Cette approche centralisee garantit que les tests restent cohérents meme quand les donnees de reference evoluent.

3.1 Identifiants de Connexion

Tous les utilisateurs de test partagent le meme mot de passe : **Admin@2026!**. Chaque utilisateur correspond a un role specifique dans le systeme RBAC, permettant de tester les differences de permissions entre les roles.

Email	Role	Organisation
admin@arpt.gn	SUPER_ADMIN	ARPT
dg@arpt.gn	DG	ARPT
dga@arpt.gn	DGA	ARPT
dir.tech@arpt.gn	DIRECTEUR_TECHNIQUE	ARPT
ing.rf@arpt.gn	INGENIEUR_RF	ARPT
analyste@arpt.gn	ANALYSTE_QOS	ARPT
auditeur@arpt.gn	AUDITEUR	ARPT
tech@orange.gn	OPERATEUR_READONLY	Orange Guinee
tech@mtn.gn	OPERATEUR_READONLY	MTN Guinee
tech@celcom.gn	OPERATEUR_READONLY	Celcom Guinee

3.2 Codes Operateurs et Regions

Operateur	Code	Region	Code
Orange Guinee	ORG	Conakry	CKY
MTN Guinee	MTN	Kindia	KND
Celcom Guinee	CEL	Boke	BOK
		Labe	LAB
		Mamou	MAM
		Faranah	FRN
		Kankan	KNK
		N'Zerekore	NZR

3.3 Payloads API Exemples

Chaque endpoint API dispose d'un payload exemple préconfiguré dans les fixtures. Ces payloads sont complets avec toutes les métriques RF, réseau, Internet et QoE pour les mesures, et avec tous les champs requis pour les alertes, campagnes, rapports, scores, utilisateurs et rôles. Les valeurs sont réalistes et correspondent aux plages attendues par les validateurs côté serveur.

4. Tests d'Authentication

Les tests d'authentification couvrent le flux complet de connexion et de gestion des sessions via NextAuth. Ils verifient que chaque utilisateur peut se connecter avec ses identifiants, que les tentatives avec des identifiants invalides sont rejetees, et que les sessions JWT sont correctement etablies. Le endpoint `/api/auth/callback/credentials` est teste avec diverses combinaisons d'email et mot de passe.

Test	Description	Statut attendu
Connexion Super Admin	POST <code>/api/auth/callback/credentials</code> avec <code>admin@arpt.gn</code>	200/302
Connexion DG	POST avec <code>dg@arpt.gn</code>	200/302
Connexion Operateur	POST avec <code>tech@orange.gn</code>	200/302
Mot de passe incorrect	POST avec mauvais mot de passe	Erreur
Email inexistant	POST avec email invalide	Erreur
Champs vides	POST avec email et password vides	Erreur
Session sans connexion	GET <code>/api/auth/session</code> sans cookie	<code>user=null</code>
Health check	GET <code>/api</code> retourne status ok	200

5. Tests API - Alertes

Les tests d'alertes couvrent les trois operations du endpoint `/api/alerts` : la lecture (GET) avec filtrage par severite, type, statut, operateur et region ; la creation (POST) d'alertes authentifiees et de signalements publics ; et la resolution/reouverture (PATCH). Les tests verifient egalement le controle d'accès RBAC, en s'assurant que les operateurs readonly ne peuvent pas resoudre d'alertes et ne voient que leurs propres alertes.

Operation	Tests	Points verifie
GET	8 tests	Liste complete, filtres (severity, type, isResolved, operateur, region, limit), 401 sans auth
POST	4 tests	Creation authentifiee, signalement public (sans auth), 400 message manquant, 401 non auth
PATCH	5 tests	Resolution, reouverture, 400 ID manquant, 500 ID inexistant, 403 operateur

6. Tests API - Mesures QoS

Les tests de mesures QoS sont les plus complets car ils couvrent trois modes de creation : la creation unitaire (POST), l'import JSON en masse (PUT), et l'import CSV en masse (PUT). Ils verifient la validation des champs requis, les plages de valeurs RF (RSSI, RSRP, RSRQ, SINR), la resolution des operateurs et regions par code, et la pagination des resultats. Le controle d'accès est également teste pour s'assurer que les operateurs ne peuvent pas creer de mesures et ne voient que leurs propres donnees.

Operation	Tests	Points verifie
GET	7 tests	Liste complete, filtres (operateur, region, type), pagination, structure, 401 sans auth
POST	6 tests	Creation mesure, champs manquants, operateur inexistant, valeur RF hors limites, 401, 403
PUT (JSON)	3 tests	Import masse, campagneld manquant, mesures vides
PUT (CSV)	3 tests	Import CSV, campagneld manquant, content-type invalide
RBAC	1 test	Operateur voit uniquement ses mesures

7. Tests API - Campagnes

Les tests de campagnes couvrent la lecture filtree et la creation de campagnes, ainsi que la mise a jour du statut via PATCH. Les statuts valides (PLANIFIEE, EN_COURS, TERMINEE, ANNULEE) sont verifiees, et les tentatives de transition vers un statut invalide sont rejetees. Les tests verifient que les operateurs readonly ne peuvent ni creer ni modifier de campagnes.

Operation	Tests	Points verifie
GET	6 tests	Liste complete, structure, filtres (operateur, statut, type), 401 sans auth
POST	5 tests	Creation, champs manquants (nom, type, operateur), 401, 403 operateur
PATCH	4 tests	Changement statut, statut invalide, ID manquant, ID inexistant

8. Tests API - Rapports

Les tests de rapports couvrent la creation, la lecture, et la mise a jour des rapports. Un point important est la distinction entre les rapports publics et prives : les utilisateurs non authentifies ne voient que les rapports marques comme publics, tandis que les administrateurs voient tous les rapports. Les statuts de rapports (PLANIFIE, EN_COURS, GENE, PUBLIE, ARCHIVE) sont verifiees lors des mises a jour.

9. Tests API - Utilisateurs, Roles et Scores

Ces tests combinent trois APIs d'administration en un seul fichier. Les tests utilisateurs couvrent la creation, la desactivation, la modification du nom, et la protection contre l'auto-desactivation. Les tests de roles couvrent la creation de nouveaux roles, la mise a jour des permissions, et la modification de la description. Les tests de scores couvrent le mecanisme d'upsert (creation ou mise a jour) et la validation des plages 0-100.

API	Operation	Tests cles
/api/users	GET	Admin voit 10+ utilisateurs, operateur 403, non auth 401
/api/users	POST	Creation utilisateur, email duplique 400, role inexistant 400
/api/users	PATCH	Modifier nom, desactiver, auto-desactivation impossible (400)
/api/roles	GET	9+ roles avec permissions, operateur 403
/api/roles	POST	Creer role + permissions, nom duplique 400, maj permissions
/api/roles	PATCH	Modifier description, ID manquant 400
/api/scores	GET	Accessible meme sans auth, filtres operateur/periode
/api/scores	POST	Upsert score, score hors limites 400, operateur inexistant 400

10. Tests API - Dashboard, Map, QoS et Audit

Les tests des APIs en lecture seule couvrent le dashboard (KPIs, operateurs, alertes, regions, SLA), la carte (donnees geographiques), les metriques QoS (agregees), le scoring (radar), et les logs d'audit. Ces endpoints sont pour la plupart accessibles sans authentification ou avec des permissions minimales, a l'exception de /api/qos et /api/audit-logs qui requierent une authentification.

11. Tests UI - Navigation et Dashboards

Les tests de navigation UI verifient le chargement de l'application, l'affichage du modal de connexion, la connexion via le formulaire, la navigation entre les onglets du dashboard (SIG, Scoring, Rapports), l'accessibilite du portail public sans connexion, et la responsivite de l'interface sur mobile (375x812) et tablette (768x1024). Ces tests sont essentiels pour garantir que l'interface utilisateur fonctionne correctement dans les differents contextes d'utilisation.

Test	Description
Chargement page d'accueil	La page se charge sans erreur, le body est visible
Login modal	Les inputs email/password sont presents sur la page d'accueil
Connexion UI	Remplir le formulaire et cliquer sur Connexion
Mauvais mot de passe	Le formulaire reste visible apres echec
Navigation onglets	Cliquer sur SIG, Scoring, Rapports sans erreur
Portail public	L'onglet Public est accessible sans connexion
Mobile (375x812)	L'interface s'adapte en mode mobile
Tablette (768x1024)	L'interface s'adapte en mode tablette

12. Tests UI - Carte Interactive SIG

Les tests de la carte interactive Leaflet verifient le chargement de la carte et de ses tuiles, le basculement entre le decoupage CNT (16 regions) et l'ancien decoupage (8 regions), l'interaction avec les polygones de prefectures et de regions (clic, popups), et le fonctionnement des controles de zoom. Ces tests sont particuliers car ils impliquent des interactions avec le composant Leaflet qui charge asynchronement.

Test	Description
Chargement carte	Leaflet container visible dans l'onglet SIG
Tuiles cartographiques	Les tuiles (tiles) se chargent correctement
Toggle CNT/Ancien	Basculer entre 16 regions CNT et 8 anciennes regions
Clic sur polygone	Un popup s'affiche au clic sur une region/prefecture
Controles de zoom	Les boutons zoom+ et zoom- fonctionnent

13. Tests RBAC - Controle d'Acces

Les tests RBAC constituent la partie la plus critique de la suite de tests E2E. Ils verifient que chaque role ne peut acceder qu'aux ressources et actions autorisees par la matrice de permissions. Les tests sont structures en quatre groupes : Super Admin (acces total), Operateur ReadOnly (acces limite), Analyste QoS (acces mesures), et Utilisateur non authentifie (acces public uniquement).

Role	Tests	Acces
SUPER_ADMIN	9 tests	Acces total a toutes les API (users, roles, alerts, campaigns, mesures, reports, audit-logs, dashboard, scores)
OPERATEUR_READONLY	10 tests	Lecture de ses alertes/campagnes/mesures/rapports publics. 403 sur users, roles, audit-logs, creation, resolution
ANALYSTE_QOS	5 tests	Lecture de mesures, alertes, rapports, campagnes, dashboard
Public (non auth)	10 tests	Dashboard, scores, rapports publics, signalement public. 401 sur users, alerts, campaigns, mesures, audit-logs

14. Fichiers de Donnees d'Import

Deux fichiers d'exemple sont fournis pour tester l'import en masse de mesures QoS. Le fichier JSON (sample-mesures-import.json) contient 4 mesures avec tous les champs optionnels remplis, pret a etre envoye via PUT /api/mesures avec le Content-Type application/json. Le fichier CSV (sample-mesures-import.csv) contient 10 mesures avec les en-tetes en anglais et francais, pret pour l'import via PUT /api/mesures avec le Content-Type text/csv.

14.1 Format JSON

```
{ "campagneId": "ID_DE_LA_CAMPAGNE", "mesures": [ { "operatorCode": "ORG", ... } ] }
```

14.2 Format CSV

```
operatorCode,regionCode,latitude,longitude,timestamp,typeMesure,rssi,rsrp,...
```

Les deux formats supportent les noms de colonnes en anglais et en francais. Le champ campagneId est obligatoire et doit etre passe soit dans le corps JSON, soit en parametre de requete pour le CSV (?campagneId=xxx).

15. Commandes d'Execution

Commande	Description
npm run test:e2e	Executer tous les tests E2E en mode headless
npm run test:e2e:headed	Executer les tests avec le navigateur visible
npm run test:e2e:ui	Ouvrir l'interface Playwright pour les tests

Commande	Description
npm run test:e2e:report	Ouvrir le rapport HTML des derniers tests
npx playwright test e2e/auth.spec.ts	Executer uniquement les tests d'authentification
npx playwright test e2e/rbac.spec.ts	Executer uniquement les tests RBAC
npx playwright test --grep 'POST'	Executer les tests contenant 'POST' dans le titre
npx playwright test --workers=2	Executer avec 2 workers (risque avec SQLite)

Avant de lancer les tests, assurez-vous que la base de données est correctement peuplée avec les données de seed (npm run db:push && npx prisma db seed). Le serveur de développement sera lancé automatiquement par Playwright si aucun serveur n'est détecté sur localhost:3000.

16. Matrice RBAC Complete

Le tableau ci-dessous presente la matrice complete des permissions par role et par endpoint API. Les codes suivants sont utilises : 200 = acces autorise, 201 = creation autorisee, 401 = authentication requise, 403 = permission insuffisante.

Endpoint	SUPER ADMIN	DG	DGA	DIR TECH	ING RF	ANAL Y STE	AUDI TEUR	OP R EAD	PUBL IC
GET /api/dashboard	200	200	200	200	200	200	200	200	200
GET /api/users	200	200	403	403	403	403	403	403	401
POST /api/users	201	403	403	403	403	403	403	403	401
PATCH /api/users	200	403	403	403	403	403	403	403	401
GET /api/roles	200	200	403	403	403	403	403	403	401
GET /api/alerts	200	200	200	200	200	200	200	200	401
POST /api/alerts	201	201	201	201	201	201	403	403	401*
PATCH /api/alerts	200	200	200	200	200	200	403	403	401
GET /api/campaigns	200	200	200	200	200	200	200	200	401
POST /api/campaigns	201	403	403	201	201	201	403	403	401
GET /api/mesures	200	200	200	200	200	200	200	200	401
POST /api/mesures	201	403	403	201	201	201	403	403	401
GET /api/reports	200	200	200	200	200	200	200	200*	200*
POST /api/reports	201	201	201	201	201	201	403	403	401
GET /api/scores	200	200	200	200	200	200	200	200	200
POST /api/scores	201	403	403	201	201	403	403	403	401
GET /api/audit-logs	200	200	200	200	403	403	403	403	401
GET /api/qos	200	200	200	200	200	200	200	200	401
GET /api/map	200	200	200	200	200	200	200	200	200

* **Notes :** POST /api/alerts avec type=SIGNALEMENT_PUBLIC est autorise sans authentication (201).
GET /api/reports pour les operateurs et le public ne retourne que les rapports marques isPublic=true. Les operateurs readonly ne voient que les donnees liees a leur propre organisation (RLS scope=own_org).